

Modelování frekvence pojistných událostí

Statistický a strojově-učící přístup na syntetickém portfoliu

Jaroslav Drobek

March 23, 2026

Abstract

Tento text prezentuje souvislý řešený příklad modelování frekvence škod na synteticky generovaném portfoliu pojištěných vozidel. Cílem je porovnat klasický lineární model, Poissonův zobecněný lineární model a moderní stromový boosting model, a to včetně validace, globální a lokální interpretace a stručného nástinu produkční pipeline.

Contents

1	Zadání úlohy	3
2	Teoretické východisko	3
2.1	Lineární model	3
2.2	Poissonův GLM	3
2.3	Boosting model	4
3	Generování syntetických dat	4
3.1	Struktura dat	4
3.2	Ukázka dat	4
3.3	Generativní schéma	5
4	Explorační analýza	5
5	Modely	5
5.1	OLS jako baseline	5
5.2	Poissonův GLM	6
5.3	XGBoost	6
6	Porovnání modelů	6
7	Validace	7
7.1	Train/test split	7
7.2	Baseline comparison	7
7.3	Overfitting check	7
7.4	Kalibrace	8

8 Interpretace modelu	8
8.1 Globální interpretace	8
8.2 Partial dependence	8
8.3 SHAP: globální interpretace	9
8.4 Lokální interpretace	10
9 Underwritingový profil	10
10 Produkční pipeline	11
11 Business použitelnost	11
12 Závěr	12
A Příloha: generování dat	13
B Příloha: trénování modelu	13
C Příloha: scoring	14
D Příloha: interpretace	14
E Příloha: metrics table	15

Cílem této studie je porovnat klasické statistické modely a moderní metody strojového učení při modelování frekvence pojistných událostí na syntetickém portfoliu. Zvláštní důraz je kladen na rovnováhu mezi predikční přesností, interpretovatelností a praktickou použitelností.

1 Zadání úlohy

Uvažujme portfolio klientů, pro něž máme k dispozici proměnné:

- věk řidiče (`age`),
- délku řidičské praxe (`experience`),
- výkon vozidla (`car_power`),
- roční nájezd (`mileage`),
- bonus–malus třídu (`bonus_malus`).

Cílovou proměnnou je počet pojistných událostí

$$Y \in \{0, 1, 2, \dots\}.$$

Úkolem je sestrojit model frekvence škod a posoudit:

1. vhodnost jednotlivých modelových tříd,
2. predikční kvalitu,
3. interpretovatelnost výsledků,
4. potenciální business použitelnost.

2 Teoretické východisko

2.1 Lineární model

Klasický lineární model předpokládá vztah

$$Y = X\beta + \varepsilon.$$

Pro count data je tento model problematický, neboť připouští i záporné predikce a není přirozeně svázán s diskrétní povahou vysvětlované veličiny.

2.2 Poissonův GLM

Přirozeným modelem pro frekvenční data je Poissonův zobecněný lineární model:

$$Y \sim \text{Poisson}(\lambda), \quad \log(\lambda) = X\beta.$$

Tento model:

- respektuje nezápornost střední hodnoty,
- poskytuje interpretovatelné koeficienty,
- tvoří standardní základ aktuárské praxe.

2.3 Boosting model

Model gradient boosting stromů umožňuje zachytit:

- nelineární vztahy,
- interakce mezi proměnnými,
- prahové a segmentové efekty.

Za cenu vyšší complexity může nabídnout lepší predikční výkon.

3 Generování syntetických dat

Byl vytvořen syntetický soubor o rozsahu $n = 5000$ pozorování. Generativní mechanismus byl zvolen tak, aby obsahoval nelinearity i realistickou variabilitu rizika.

3.1 Struktura dat

Použité proměnné:

Proměnná	Význam
age	věk řidiče
experience	délka praxe
car_power	výkon vozidla
mileage	roční nájezd
bonus_malus	tarifní třída
claims	počet škod

3.2 Ukázka dat

Data byla synteticky generována tak, aby realisticky odpovídala typickému portfoliu pojištění motorových vozidel. Použití syntetických dat umožňuje plnou kontrolu nad strukturou rizika a současně eliminuje omezení spojená s ochranou osobních údajů.

Table 1: Ukázka synteticky generovaných dat

age	experience	car _{power}	mileage	bonus _{malus}	claims
56	32	92.810610	20230.566569	4	0
69	42	137.871259	11872.708437	3	2
46	19	66.422355	22919.942917	0	3
32	11	194.875500	19872.900523	4	3
60	35	76.217748	13120.602744	4	4
25	2	147.204548	5634.287201	2	4
78	57	86.818169	23753.757032	3	2
38	17	157.634207	10368.841646	4	1
56	37	166.523205	25861.843587	4	5
75	56	151.012959	15507.256501	4	0

3.3 Generativní schéma

Intenzita škod byla modelována ve tvaru

$$\lambda = \exp(f(x)),$$

kde $f(x)$ je nelineární funkce prediktorů. Samotný počet škod byl následně generován z Poissonova rozdělení.

4 Explorační analýza

V první fázi byly zkoumány:

- základní popisné statistiky,
- rozdělení cílové proměnné,
- vztah mezi průměrem a variancí,
- jednoduché grafické závislosti.

Poznámka 4.1. Pro count data je klíčové ověřit, zda se jedná o silně asymetrické rozdělení s převahou nulových hodnot, což je pro frekvenční úlohy typické.

5 Modely

5.1 OLS jako baseline

Lineární model byl použit jako referenční baseline. Jeho role je především didaktická: ukazuje, proč count data vyžadují vhodnější modelovou třídu.

Zde je vhodné vložit stručnou tabulku koeficientů nebo zkrácený komentář k výsledkům OLS.

5.2 Poissonův GLM

Poissonův GLM byl odhadnut ve tvaru

$$\log(\lambda) = \beta_0 + \beta_1 \text{age} + \beta_2 \text{mileage} + \beta_3 \text{car_power} + \beta_4 \text{bonus_malus} + \beta_5 \text{experience}.$$

Po exponenciaci koeficientů

$$\exp(\beta_j)$$

lze parametry interpretovat jako multiplikativní změny očekávané frekvence.

5.3 XGBoost

Boosting model byl trénován s Poisson objective. Tím vznikl model schopný zachytit nelineární a interakční efekty, které GLM bez dalšího rozšíření postihuje jen omezeně.

6 Porovnání modelů

Jednotlivé modely byly porovnány z hlediska predikční přesnosti a praktické použitelnosti. Vedle absolutní chyby (MAE) byly sledovány také podíly přesné shody po zaokrouhlení predikce a podíly případů, v nichž se predikce od skutečnosti lišila nejvýše o jednu škodu.

Table 2: Souhrn základních metrik modelů

Metric	Value
Mean claims	1.9646
Variance claims	2.5935
Baseline MAE (full sample)	1.2350
OLS MAE	1.0976
GLM Poisson MAE	1.0858
XGBoost MAE	1.0570
OLS exact match	0.2796
GLM exact match	0.2844
XGB exact match	0.2920
OLS within ± 1	0.7334
GLM within ± 1	0.7474
XGB within ± 1	0.7618
Baseline MAE (test)	1.2133
XGB train MAE	1.0553
XGB test MAE	1.0740
XGB test/train ratio	1.0177

Průměrný počet škod na klienta činí přibližně 1.96 při varianci 2.59, což odpovídá středně rizikovému portfoliu bez extrémní dominance nulových pozorování. V takovém kontextu je

absolutní chyba modelu přirozeně řádu jedné pojistné události.

Triviální baseline model predikující konstantní průměr dosahuje MAE přibližně 1.24. Všechny uvažované modely tuto baseline jednoznačně překonávají, což potvrzuje, že využívají informaci obsaženou ve vysvětlujících proměnných.

Lineární model OLS poskytuje zlepšení oproti baseline, avšak jeho použití na početní data je metodicky omezené. Poissonův GLM představuje statisticky přirozenější přístup a vykazuje mírně lepší výsledky, přičemž zachovává vysokou interpretovatelnost parametrů.

Nejnižší chybu dosahuje model XGBoost, který dokáže zachytit nelineární závislosti a interakce mezi proměnnými. Výsledky přesné shody a shody v toleranci ± 1 rovněž ukazují jeho mírnou převahu nad klasickými modely.

Z praktického hlediska je důležité, že rozdíly mezi GLM a XGBoostem nejsou dramatické. To odpovídá zkušenosti z aplikované praxe, kde GLM často poskytuje silnou a transparentní baseline, zatímco strojově-učící modely fungují jako výkonnější, avšak komplexnější alternativa.

Výsledky naznačují, že moderní metody strojového učení mohou přinést určité zlepšení predikční přesnosti, avšak klasické GLM zůstává silným a transparentním referenčním přístupem.

7 Validace

7.1 Train/test split

Data byla rozdělena na trénovací a testovací část v poměru 70:30. Model byl natrénován pouze na trénovací množině a následně vyhodnocen na nezávislé testovací sadě, což umožňuje posoudit jeho skutečnou generalizační schopnost.

7.2 Baseline comparison

Výkon modelu byl porovnán s triviální baseline predikující konstantní hodnotu rovnou průměrnému počtu škod v datech. Požadovaným minimem je překonání této baseline.

Model XGBoost dosahuje na testovací množině nižší absolutní chyby než baseline, což potvrzuje, že využívá informaci obsaženou ve vysvětlujících proměnných a poskytuje skutečně informativní predikce.

7.3 Overfitting check

Poměr testovací a trénovací chyby vyšel přibližně

$$\frac{\text{MAE}_{test}}{\text{MAE}_{train}} \approx 1.02.$$

Takto nízká hodnota indikuje velmi dobrou generalizaci a absenci významného přeučení. Model si zachovává podobnou úroveň přesnosti na známých i dosud neviděných datech.

7.4 Kalibrace

Kalibrační analýza byla provedena po decilech predikovaného rizika. Body kalibrační křivky ležely v blízkosti diagonály, což značí, že model je v průměru dobře sladěn se skutečným počtem škod a správně reprodukuje relativní úroveň rizika napříč portfoliem.

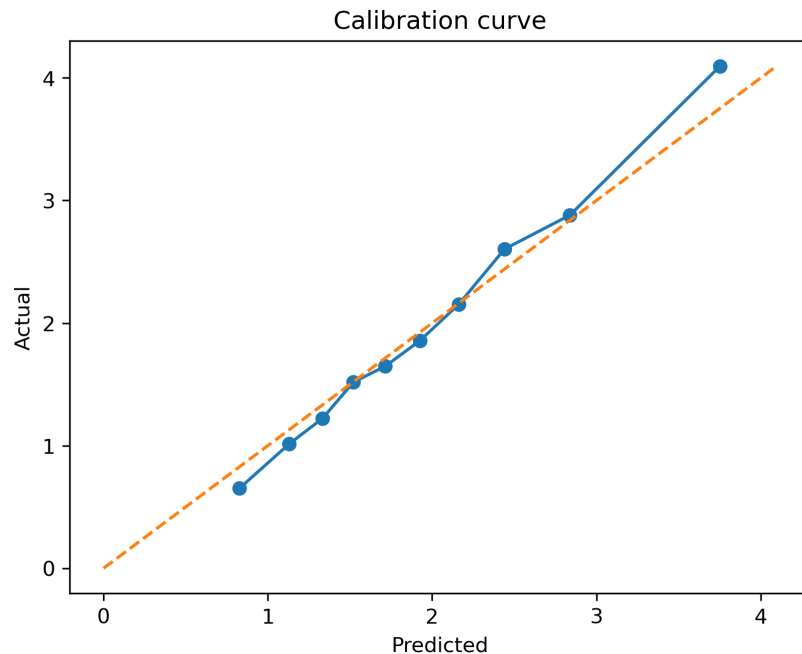


Figure 1: Kalibrační křivka modelu

8 Interpretace modelu

8.1 Globální interpretace

SHAP analýza poskytuje lokální vysvětlení jednotlivých predikcí, které doplňuje globální pohled daný parametry lineárních modelů.

Globální interpretace odpovídá otázce:

Jak model funguje na úrovni celé populace?

Použité nástroje:

- feature importance,
- partial dependence plots,
- SHAP summary plot.

8.2 Partial dependence

Partial dependence grafy ukázaly:

- téměř monotónní růst rizika s nájezdem,

- nelineární chování u výkonu vozidla,
- slabší a lokálně proměnlivý efekt věku.

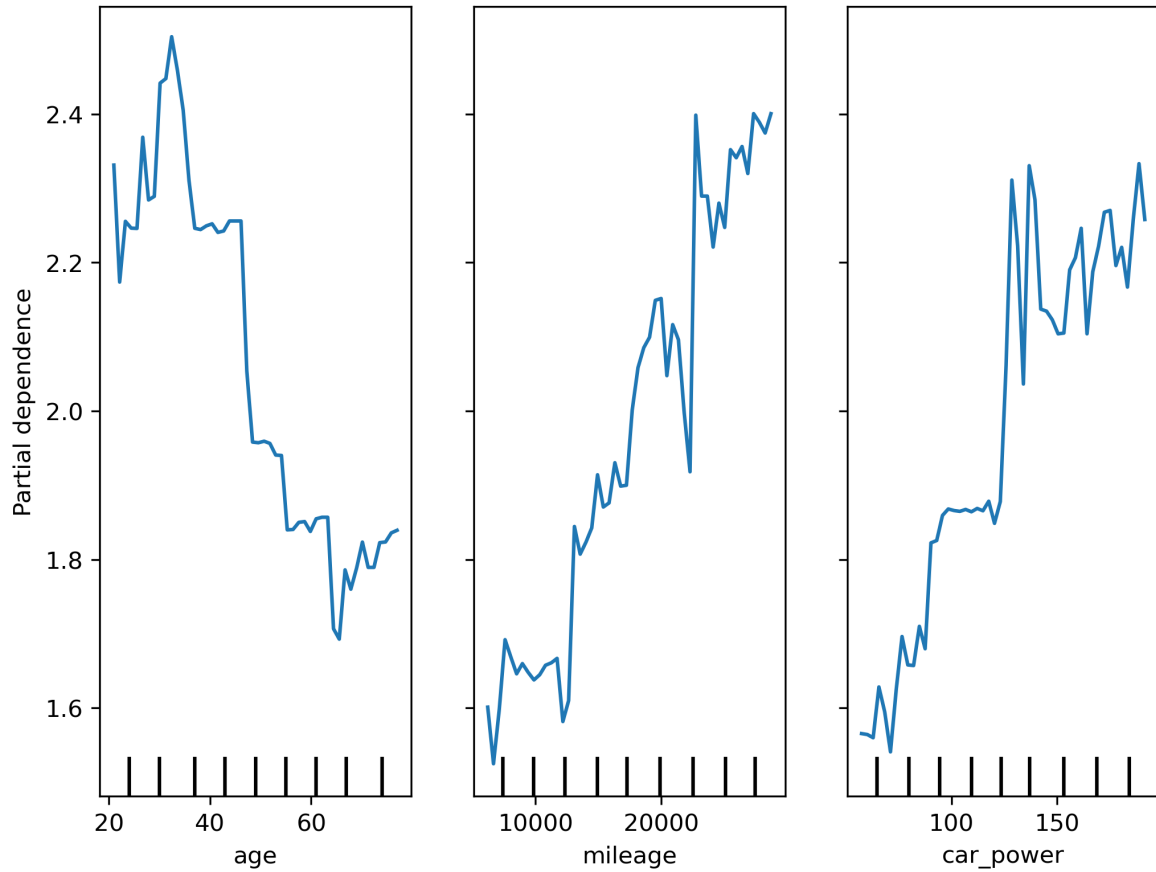


Figure 2: Grafy dílčích závislostí vybraných proměnných

8.3 SHAP: globální interpretace

SHAP summary plot umožnil seřadit proměnné podle jejich významu a zároveň pozorovat:

- směr vlivu,
- nonlinearity,
- interakce.

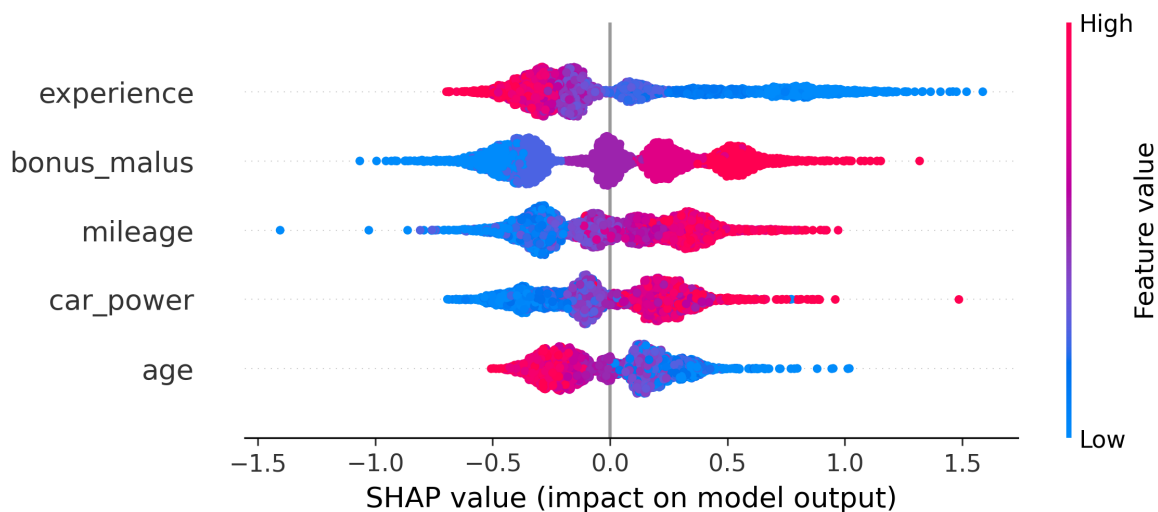


Figure 3: Globální SHAP interpretace

8.4 Lokální interpretace

Lokální interpretace odpovídá otázce:

Proč model rozhodl právě takto pro konkrétního klienta?

K tomu byly použity:

- SHAP waterfall plot,
- underwriting profil klienta.

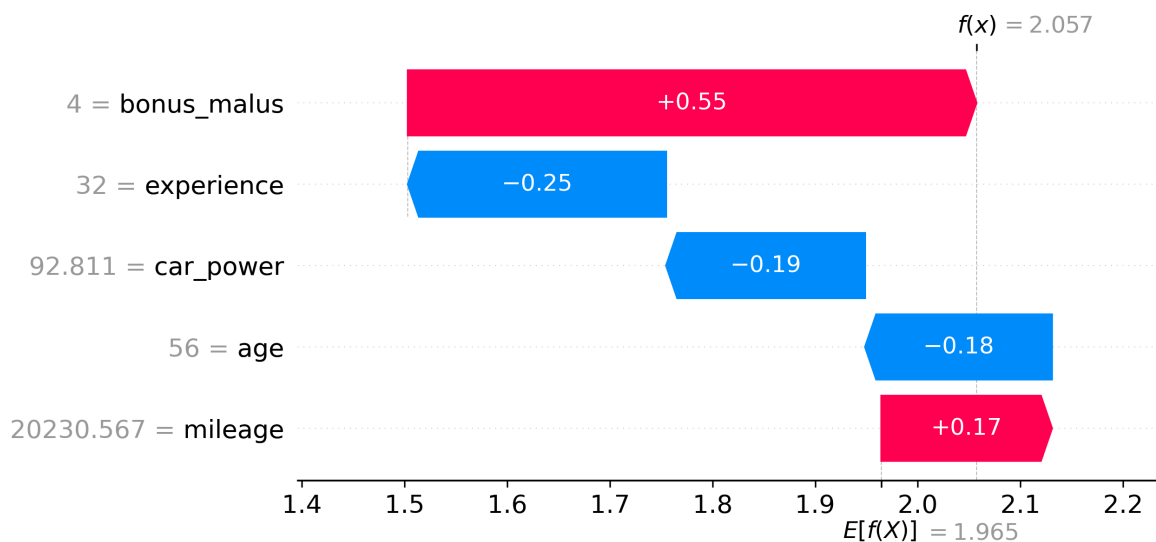


Figure 4: Lokální SHAP interpretace pro vybraného klienta

9 Underwritingový profil

Na základě lokálních SHAP hodnot lze vytvořit stručný underwriting komentář.

Příklad 9.1. Vybraný klient má predikovanou frekvenci škod 0.200 oproti portfoliovému průměru 0.171, tj. přibližně $1.17\times$ vyšší riziko. Hlavními faktory zvyšujícími riziko jsou výkon vozidla a bonus–malus, zatímco nájezd působí v daném případě riziko snižujícím směrem.

10 Produkční pipeline

Notebook představuje výzkumnou a explorační vrstvu. V produkčním prostředí by byl model rozdělen na:

- trénovací skript,
- ukládání modelového artefaktu,
- samostatný scoring skript,
- validaci a monitoring.

Schematicky:

data $\rightarrow$ preprocessing $\rightarrow$ model $\rightarrow$ validace $\rightarrow$ scoring $\rightarrow$ report.

11 Business použitelnost

Model je businessově použitelný pouze tehdy, pokud:

1. je přesnější než jednoduché baseline,
2. generalizuje na nová data,
3. je dostatečně interpretovatelný,
4. vede k realizovatelnému rozhodnutí.

Možné oblasti použití:

- pricing,
- segmentace portfolia,
- underwriting,
- identifikace rizikových skupin.

Současně platí, že v regulovaném prostředí může mít GLM stále výhodu vyšší transparentnosti, zatímco ML model plní roli challengeru nebo doplňkového nástroje.

12 Závěr

Předložený příklad ukazuje celý životní cyklus analytického úkolu:

zadání → data → EDA → model → validace → interpretace → pipeline → business závěr.

Poissonův GLM poskytuje přirozený a dobře interpretovatelný základ pro modelování frekvence škod. XGBoost oproti tomu umožňuje zachytit nelineární a interakční struktury, a tím potenciálně zvýšit predikční výkon. Kombinace obou přístupů — statistického a strojově-učícího — se v aplikované praxi ukazuje jako mimořádně silná.

Studie ukazuje, že při modelování frekvence škod mohou metody strojového učení nabídnout vyšší predikční přesnost, avšak za cenu nižší interpretovatelnosti. Poissonův GLM zůstává robustním a transparentním nástrojem, zatímco modely typu XGBoost představují vhodný doplněk pro úlohy, kde je prioritou čistá predikční výkonnost.

A Příloha: generování dat

Zde je vhodné vložit zkrácený Python kód generování dat.

Listing 1: Skeleton generování syntetických dat

```
np.random.seed(42)
n = 5000

age = np.random.randint(18, 80, n)
experience = np.clip(age - np.random.randint(18, 30, n), 0, None)
car_power = np.random.uniform(50, 200, n)
mileage = np.random.uniform(5000, 30000, n)
bonus_malus = np.random.randint(0, 5, n)

risk = (
    0.02 * (age - 40) ** 2 / 100
    + 0.00002 * mileage
    + 0.003 * car_power
    - 0.02 * experience
    + 0.15 * bonus_malus
)

claims = np.random.poisson(np.exp(risk))

df = pd.DataFrame({
    "age": age,
    "experience": experience,
    "car_power": car_power,
    "mileage": mileage,
    "bonus_malus": bonus_malus,
    "claims": claims
})

X = df.drop(columns="claims")
y = df["claims"]
```

B Příloha: trénování modelu

Listing 2: Skeleton trénování modelu

```
model = XGBRegressor(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.05,
    random_state=42
)
model.fit(X, y)
```

C Příloha: scoring

Listing 3: Skeleton scoring

```
import joblib
import pandas as pd

model = joblib.load("models/xgb_frequency_model.joblib")

new_client = pd.DataFrame([
    "age": 45,
    "experience": 20,
    "car_power": 120,
    "mileage": 15000,
    "bonus_malus": 1
])

prediction = model.predict(new_client)

print("Predikovan po et kod :", prediction[0])
```

D Příloha: interpretace

Listing 4: Skeleton interpretace

```
model = joblib.load("models/xgb_frequency_model.joblib")

pred = model.predict(X)

# KALIBRACE
bins = pd.qcut(pred, 10, duplicates="drop")
cal = pd.DataFrame({"pred": pred, "actual": y, "bin": bins})
cal = cal.groupby("bin").mean()

plt.figure()
plt.plot(cal["pred"], cal["actual"], "o-")
plt.plot([0, cal.values.max()], [0, cal.values.max()], "--")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Calibration curve")

plt.savefig(f"{FIG_DIR}/calibration.png",
            dpi=300, bbox_inches="tight")
plt.close()

# PDP
features = ["age", "mileage", "car_power"]
```

```

fig, ax = plt.subplots(figsize=(8, 6))
PartialDependenceDisplay.from_estimator(
    model, X, features, grid_resolution=50, ax=ax
)

plt.savefig(f"{FIG_DIR}/pdp.png",
            dpi=300, bbox_inches="tight")
plt.close()

# SHAP
explainer = shap.Explainer(model)
shap_values = explainer(X)

# summary plot
shap.summary_plot(shap_values, X, show=False)
plt.savefig(f"{FIG_DIR}/shap_summary.png",
            dpi=300, bbox_inches="tight")
plt.close()

# waterfall pro prvn ho klienta
shap.plots.waterfall(shap_values[0], show=False)
plt.savefig(f"{FIG_DIR}/shap_waterfall_client0.png",
            dpi=300, bbox_inches="tight")
plt.close()

```

E Příloha: metrics table

Listing 5: Skeleton scoring

```

# Datasetsy pro modely
X_ml = df[["age", "mileage", "car_power", "bonus_malus", "experience"
]].astype(float)
y = df["claims"]
X_glm = sm.add_constant(X_ml)

# Baseline
mean_claims = y.mean()
var_claims = y.var()

baseline_pred = np.full(len(y), mean_claims)
baseline_mae = mean_absolute_error(y, baseline_pred)

# OLS
ols_model = sm.OLS(y, X_glm).fit()
pred_ols = ols_model.predict(X_glm)

```

```

# GLM Poisson
glm_model = sm.GLM(y, X_glm, family=sm.families.Poisson()).fit()
pred_glm = glm_model.predict(X_glm)

# XGB full-sample (pro srovnání modelů)
xgb_full = XGBRegressor(
    objective="count:poisson",
    n_estimators=200,
    max_depth=3,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)
xgb_full.fit(X_ml, y)
pred_xgb = xgb_full.predict(X_ml)

# Zaokrouhlení predikce
rounded_ols = np rint(np.clip(pred_ols, 0, None)).astype(int)
rounded_glm = np rint(np.clip(pred_glm, 0, None)).astype(int)
rounded_xgb = np rint(np.clip(pred_xgb, 0, None)).astype(int)

# In-sample metriky
mae_ols = mean_absolute_error(y, pred_ols)
mae_glm = mean_absolute_error(y, pred_glm)
mae_xgb = mean_absolute_error(y, pred_xgb)

exact_ols = (rounded_ols == y).mean()
exact_glm = (rounded_glm == y).mean()
exact_xgb = (rounded_xgb == y).mean()

within1_ols = (np.abs(rounded_ols - y) <= 1).mean()
within1_glm = (np.abs(rounded_glm - y) <= 1).mean()
within1_xgb = (np.abs(rounded_xgb - y) <= 1).mean()

# Train/test validace pro XGB
X_train, X_test, y_train, y_test = train_test_split(
    X_ml, y, test_size=0.3, random_state=42
)

xgb_tt = XGBRegressor(
    objective="count:poisson",
    n_estimators=200,
    max_depth=3,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,

```



```

    random_state=42
)
xgb_tt.fit(X_train, y_train)

pred_train = xgb_tt.predict(X_train)
pred_test = xgb_tt.predict(X_test)

mae_train = mean_absolute_error(y_train, pred_train)
mae_test = mean_absolute_error(y_test, pred_test)
test_train_ratio = mae_test / mae_train

baseline_test_pred = np.full(len(y_test), y_train.mean())
baseline_test_mae = mean_absolute_error(y_test, baseline_test_pred)

# Tabulka metrik
metrics = pd.DataFrame([
    ("Mean claims", mean_claims),
    ("Variance claims", var_claims),
    ("Baseline MAE (full sample)", baseline_mae),
    ("OLS MAE", mae_ols),
    ("GLM Poisson MAE", mae_glm),
    ("XGBoost MAE", mae_xgb),
    ("OLS exact match", exact_ols),
    ("GLM exact match", exact_glm),
    ("XGB exact match", exact_xgb),
    ("OLS within 1 ", within1_ols),
    ("GLM within 1 ", within1_glm),
    ("XGB within 1 ", within1_xgb),
    ("Baseline MAE (test)", baseline_test_mae),
    ("XGB train MAE", mae_train),
    ("XGB test MAE", mae_test),
    ("XGB test/train ratio", test_train_ratio),
], columns=["Metric", "Value"])

metrics.to_latex(
    os.path.join(TAB_DIR, "metrics_summary.tex"),
    index=False,
    float_format="%.4f",
    escape=False
)

```